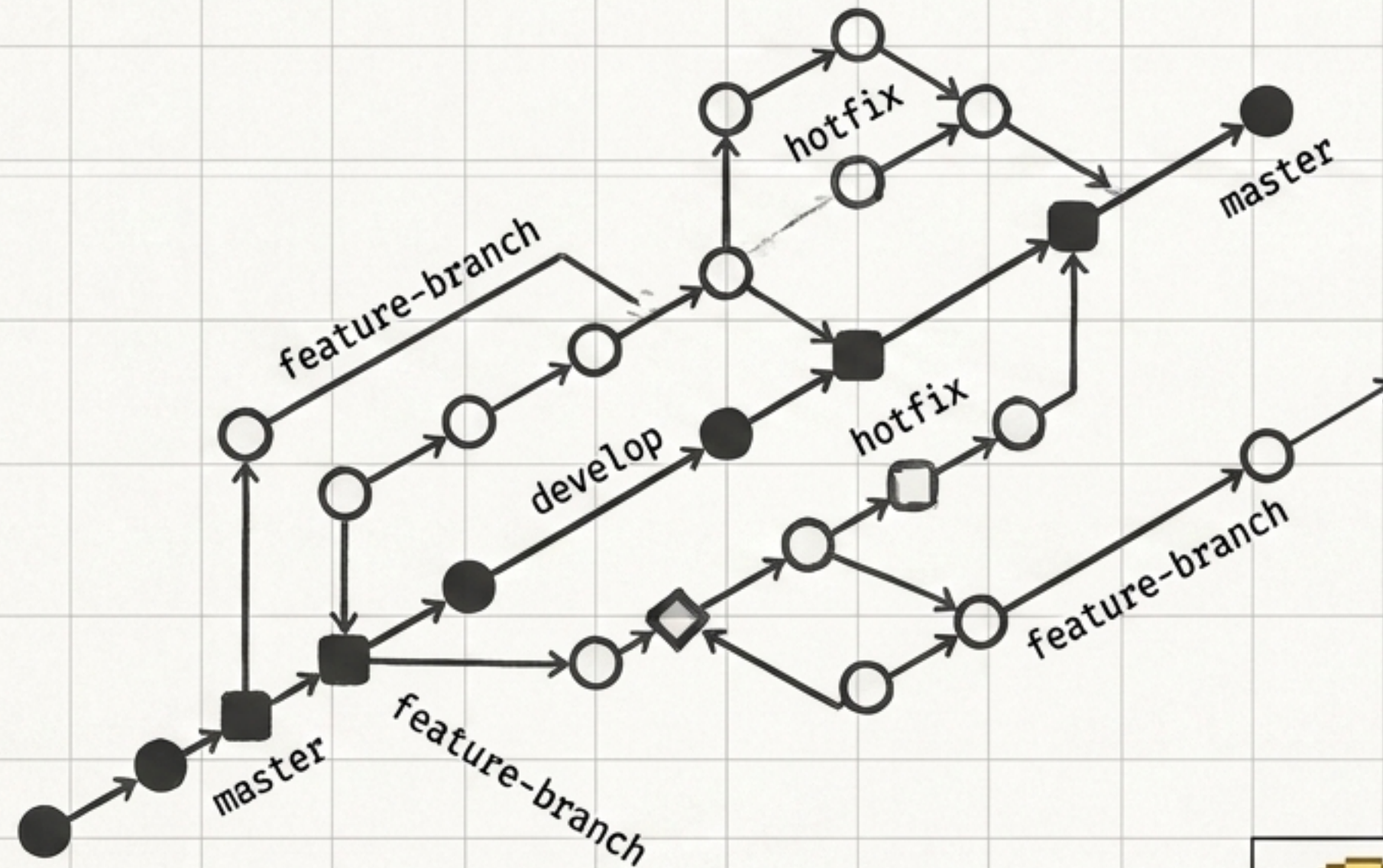


[TITLE]: The Ultimate Git Visual Map

[SUBTITLE]: สรุปทุกคำสั่งและสถาปัตยกรรม Git ฉบับพิมพ์เขียว

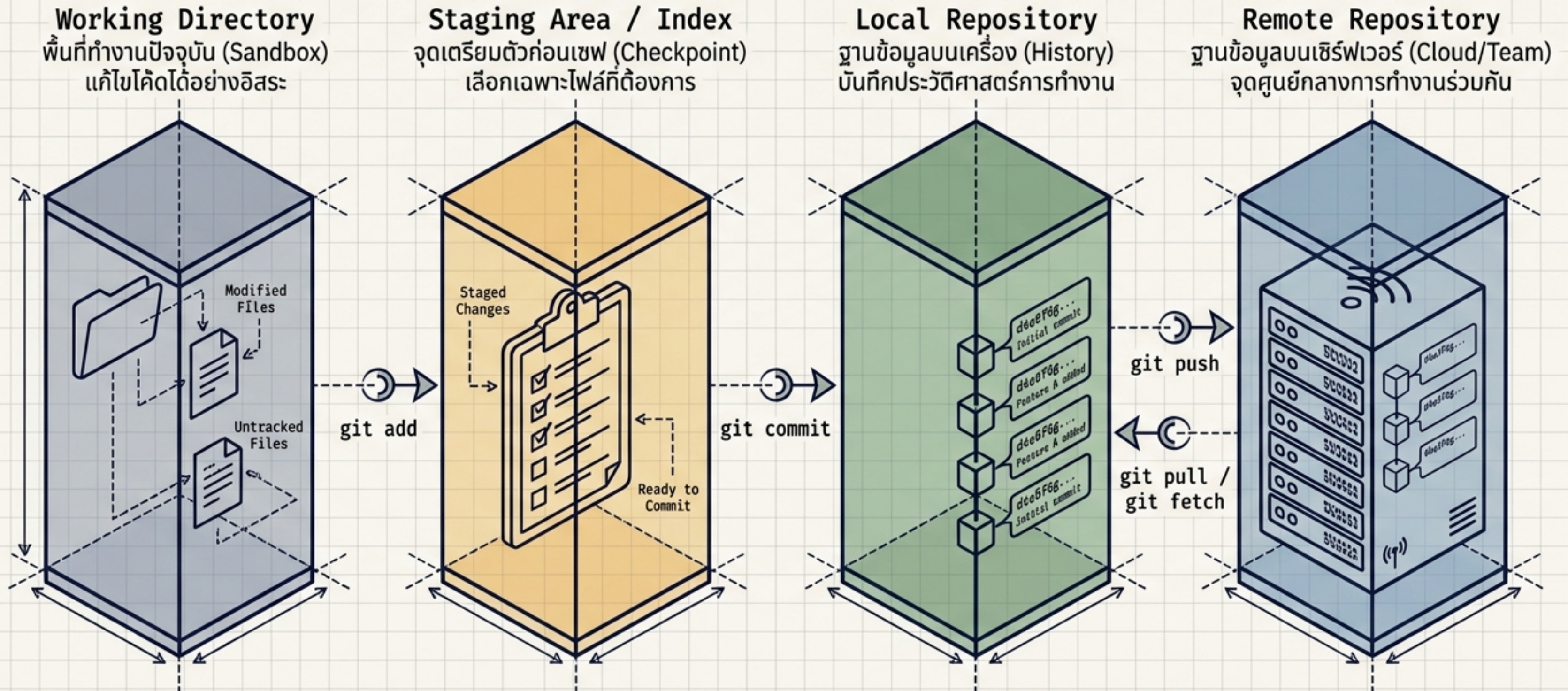
[TAGLINE]: เปลี่ยนระบบควบคุมเวอร์ชันที่ซับซ้อน ให้กลายเป็นภาพจำที่ใช้งานได้จริง



[FOOTNOTE]:
Based on the Git Cheat Sheet
by Flavio Copes (2024)

[HEADING]: The Core Mental Model: สถาปัตยกรรม 4 พื้นที่

[BODY]: คำสั่ง Git เกือบทั้งหมดคือยานพาหนะที่ใช้ย้ายข้อมูลข้ามไปมาระหว่าง 4 พื้นที่นี้
การจำคำสั่งได้ไม่สำคัญเท่าการรู้ว่าตอนนี้เรากำลังทำงานอยู่ตรงไหน



Foundation: การตั้งค่าเริ่มต้นและปรับแต่ง Workspace

Initialization: เริ่มต้นโปรเจกต์ใหม่

```
git init
สร้างโฟลเดอร์ .git เพื่อเริ่ม Track ไฟล์
```

```
git clone <url>
คัดลอก Repository จาก Remote มาไว้ที่ Local
```

Identity & Security: ยืนยันตัวตนและความปลอดภัย

```
git config --global user.name / user.email
ตั้งค่าชื่อและอีเมลประจำตัว
```

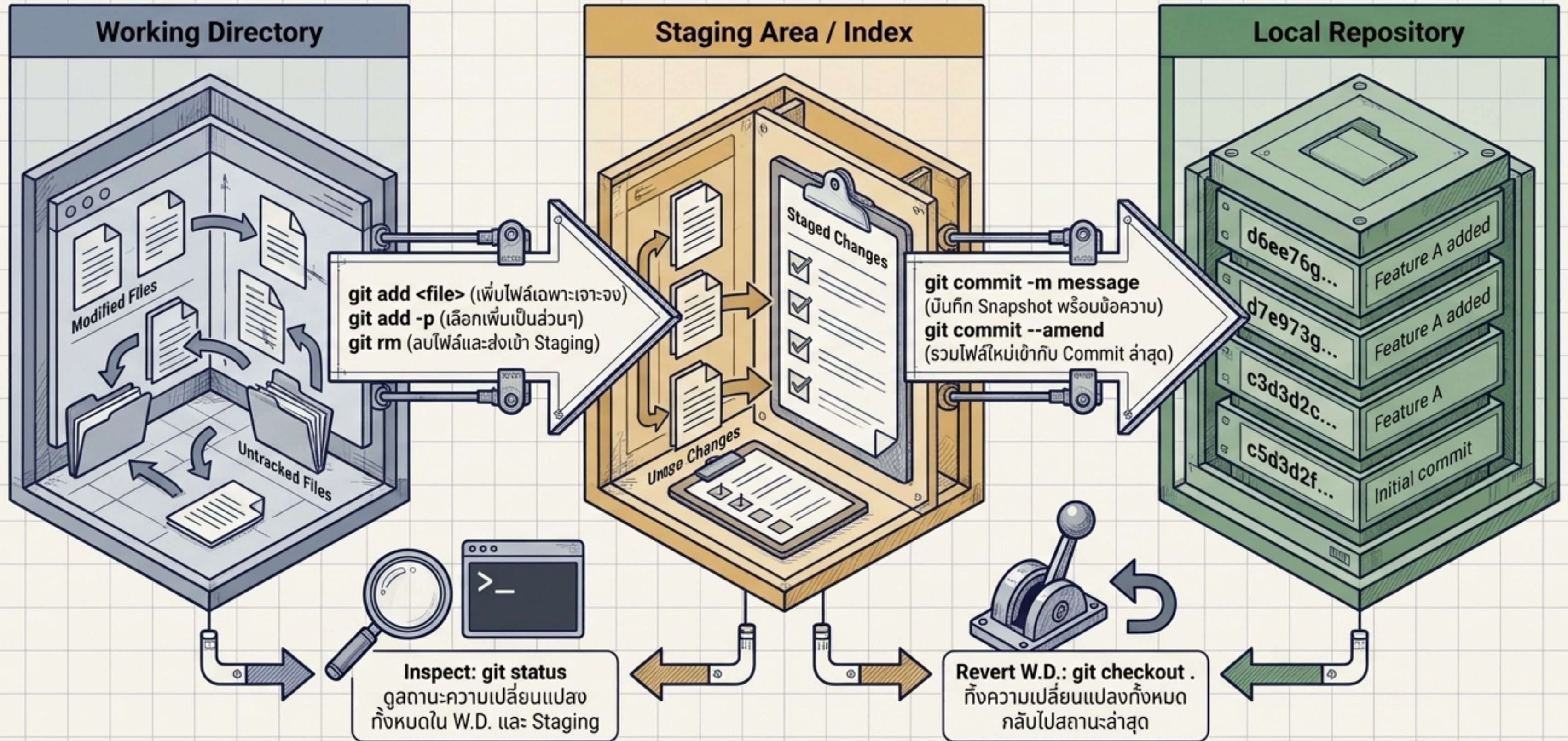
```
git config --global commit.gpgSign true
บังคับเซ็น GPG Sign ทุก Commit เพื่อยืนยันความถูกต้อง
(Data Integrity)
```

Aliases: สร้างคีย์ลัดเพิ่มความเร็ว

```
git config --global alias.st status
(เปลี่ยนคำสั่ง status ให้เหลือแค่ st)
```

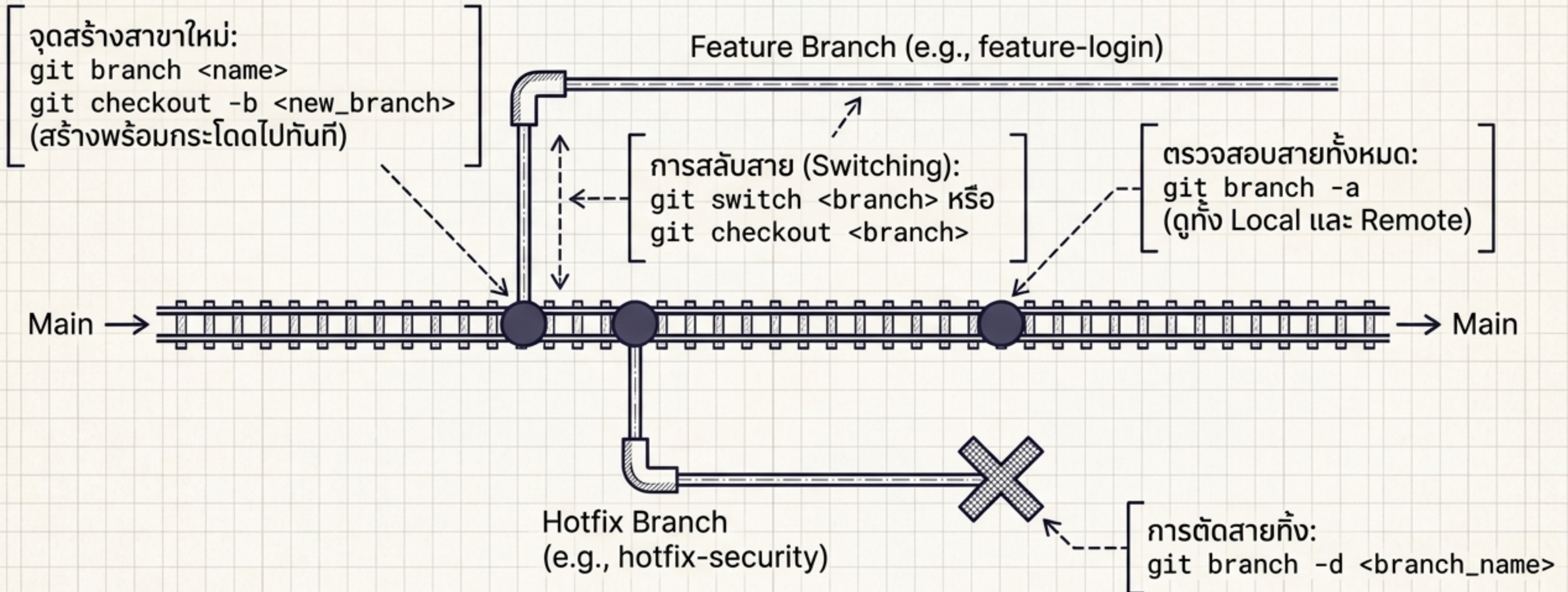
```
git config --global alias.co checkout
git config --global alias.br branch
```


The Commit Lifecycle: วงจรชีวิตการบันทึกโค้ด



Branching Architecture: มิติคู่ขนานของการพัฒนา

Branch ช่วยให้เราทดลองโค้ดและพัฒนาฟีเจอร์ได้อย่างอิสระ โดยไม่กระทบต่อโค้ดหลัก จนกว่าจะทำการรวม (Merge)



Merging Strategies: ศิลปะแห่งการรวมมิตี

Default / Fast-Forward

Network Graph After Merge

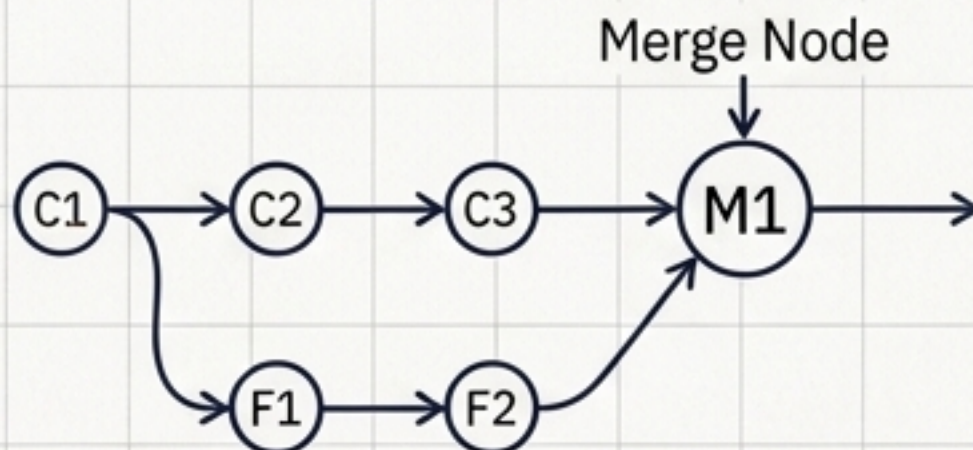


git merge <branch>

รวมประวัติเข้าด้วยกันแบบปกติ
ปลายทางขยับไปที่จุดล่าสุด

No Fast-Forward

Network Graph After Merge

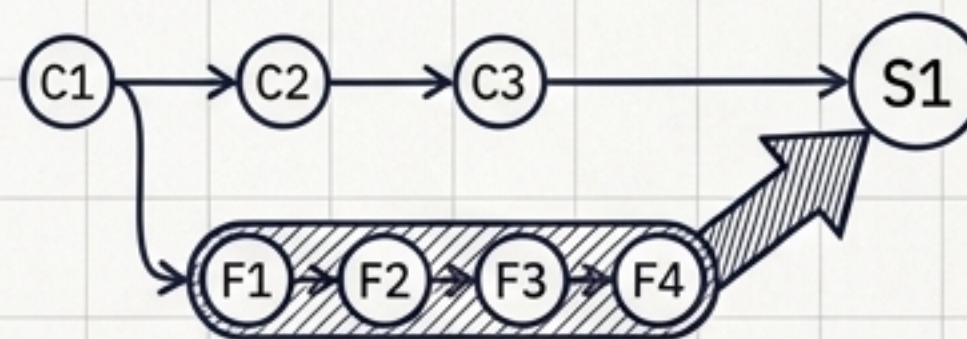


git merge --no-ff <branch>

บังคับสร้าง Merge Commit ใหม่เสมอ
ช่วยรักษาร่องรอยว่าฟีเจอร์นี้ถูกรวมเข้ามาเมื่อไหร่

Squash

Network Graph After Merge



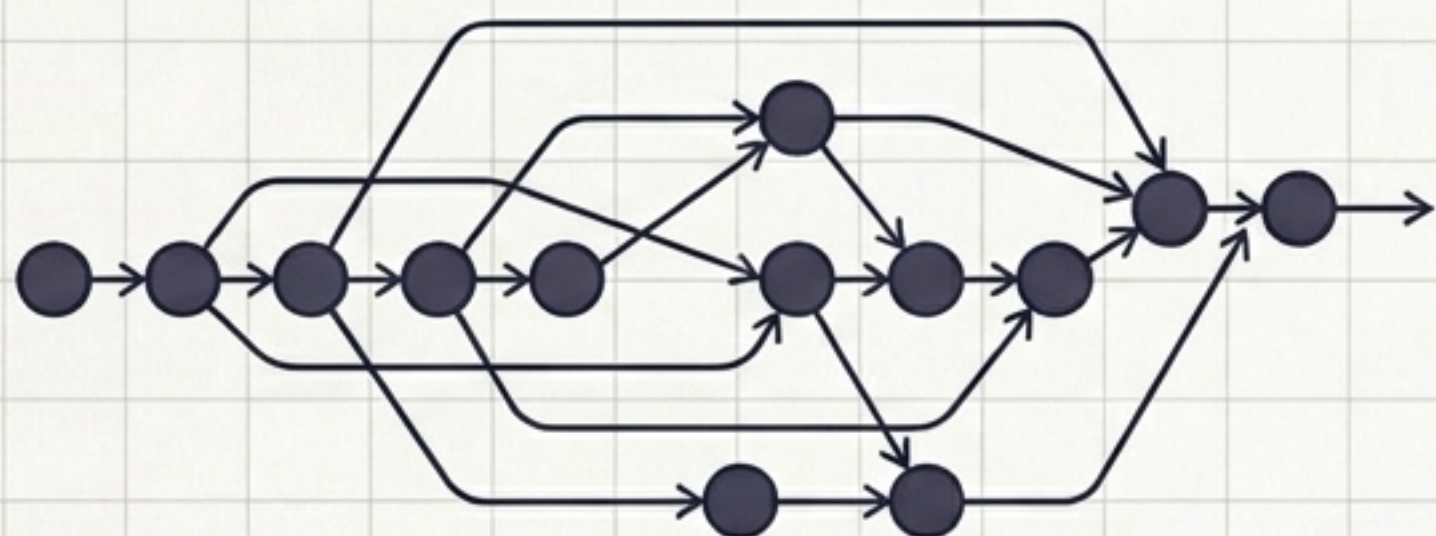
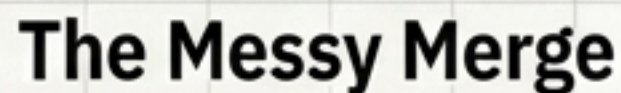
git merge --squash <branch>

ยุบรวม Commits ทั้งหมดจาก Branch สอง
ให้กลายเป็น Commit เดียว (ประวัติสะอาด)

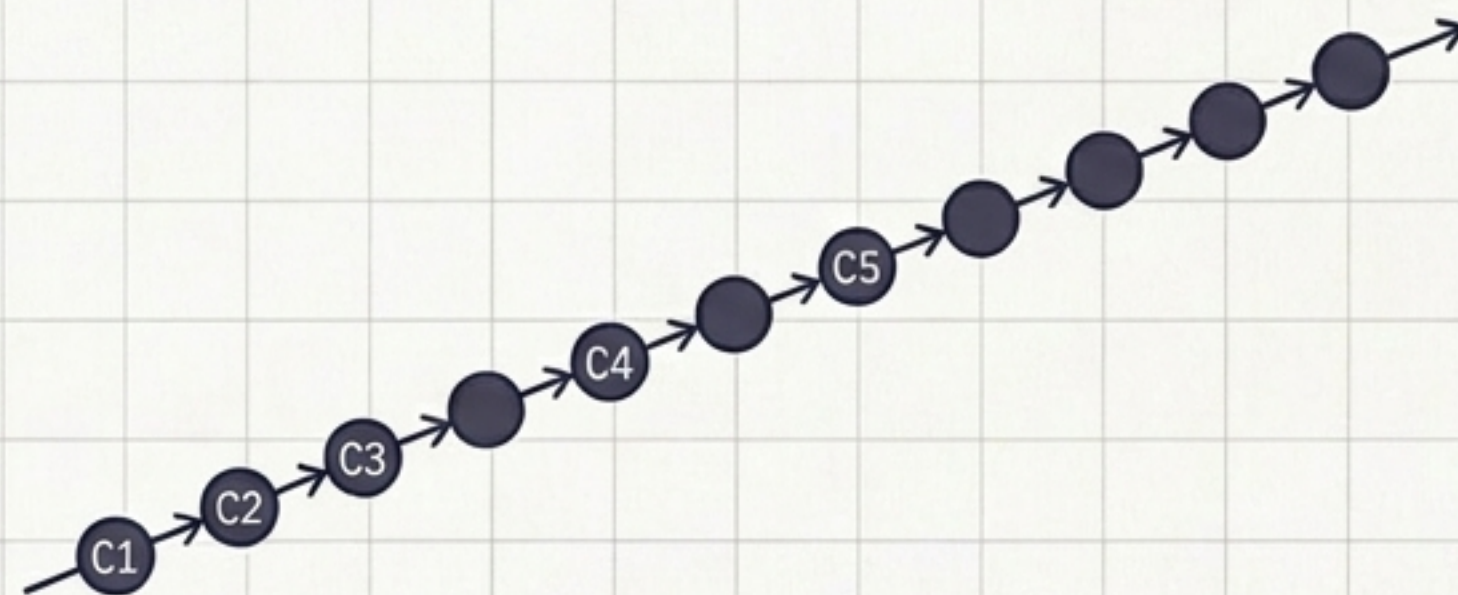
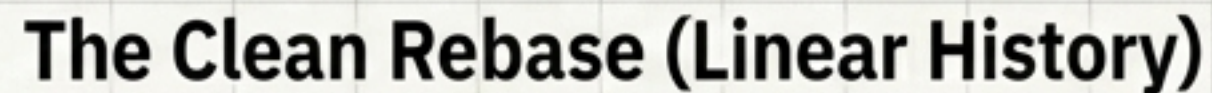
เจอข้อผิดพลาดชนกัน (Conflict)? ใช้ git merge --abort เพื่อยกเลิกและถอยกลับจุดเริ่มต้น

Rebasing: การจัดเรียงประวัติศาสตร์ใหม่ให้เป็นเส้นตรง

ดึง Commit ของเราไปต่อท้ายปลายทางของ Branch หลัก เสมือนว่าเราเพิ่งเขียนโค้ดต่อจากอัปเดตล่าสุด



Merge ธรรมดาที่เกิดการซ้อนทับกันมากเกินไป



Linear History (ประวัติเรียงลำดับอย่างเป็นระเบียบ)

```
git rebase <branch>
```

ย้ายฐานประทิตใหม่ให้เป็นเส้นตรง

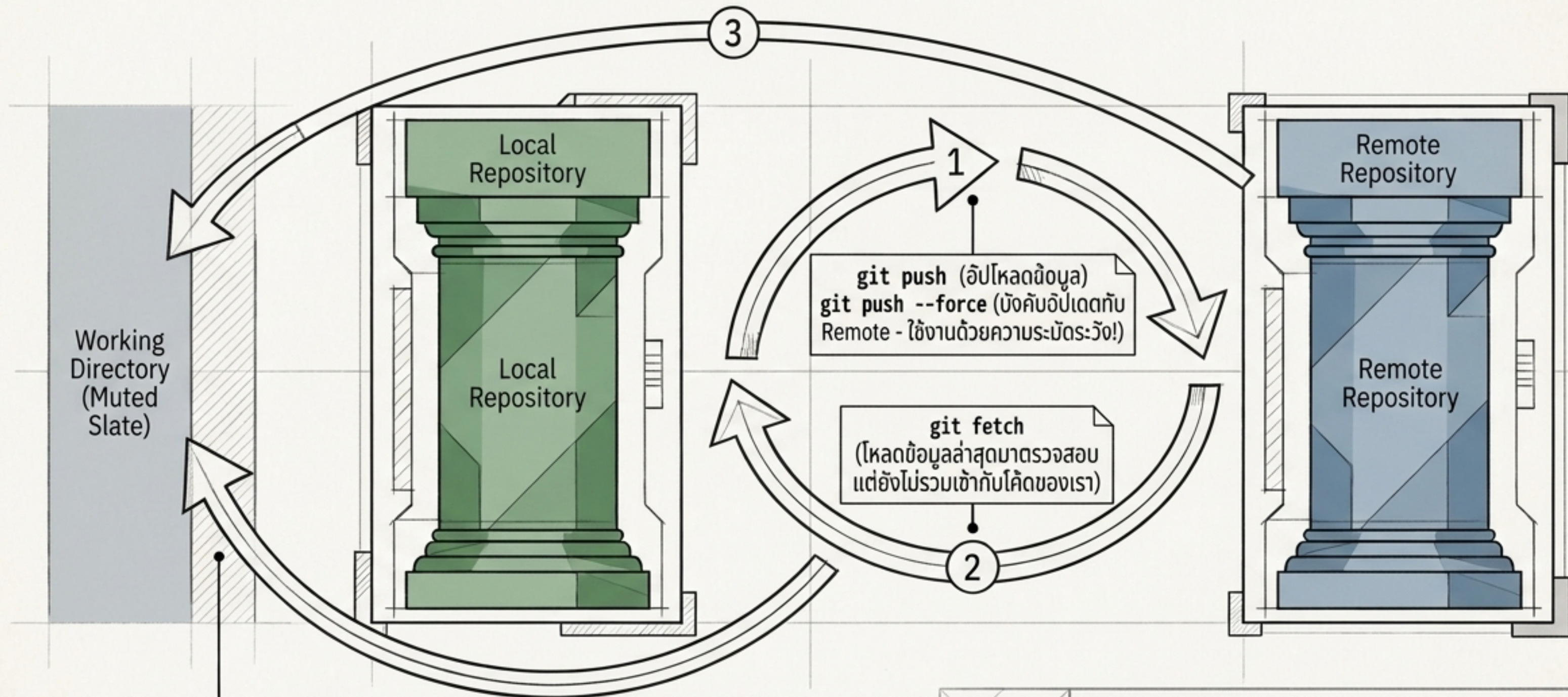
```
git rebase -i <branch>
```

Interactive Rebase: เปิดหน้าต่างเพื่อยุบรวม (Squash) แก้ไข หรือลบ Commit ที่ก่อนหน้านี้ไปต่อท้าย

```
git rebase --continue / --abort
```

ทำต่อเมื่อแก้ Conflict เสร็จ
หรือยกเลิกทั้งหมด

Remote Collaboration: ชิงค์ข้อมูลกับทีม



git push (อัปโหลดข้อมูล)
git push --force (บังคับอัปเดตกับ Remote - ใช้งานด้วยความระมัดระวัง!)

git fetch
(โหลดข้อมูลล่าสุดมาตรวจสอบ แต่ยังไม่รวมเข้ากับโค้ดของเรา)

git pull
(= fetch + merge) โหลดข้อมูลพร้อมรวมเข้า W.D.
(แนะนำให้ใช้ `git pull --rebase` เพื่อป้องกัน Merge Commit ชะงัก)



Network Config:

การจัดการ Remotes:

`git remote -v` (ดูรายการ URL ทั้งหมด)

`git remote add upstream <url>` (ตั้งเป็น Remote สำหรับโปรเจกต์ที่ Fork มา)

The Undo Matrix: คู่มือย้อนเวลาเมื่อเกิดข้อผิดพลาด

X-Axis Label
ผลลัพธ์ต่อ History (สร้างประวัติใหม่กับ vs ทำลายอดีตทิ้ง)

Y-Axis Label
ผลลัพธ์ต่อ W.D.
(รักษางานเดิมไว้ vs ล้างบาง W.D.)

Quadrant 1

git revert <commit>

[ปลอดภัยที่สุด] สร้าง Commit ใหม่ที่ลบผลลัพธ์ของ Commit เก่า โดยไม่แตะต้องประวัติเดิม

Quadrant 2

git reset --soft <commit>

ย้อน HEAD กลับไปอดีต แต่เก็บไฟล์ที่ทำมาไว้ใน Staging (พร้อม Commit ใหม่)

Quadrant 3

git reset --mixed <commit>

(Default) ย้อน HEAD ทั้ง Staging แต่เก็บโค้ดทั้งหมดไว้ใน Working Directory

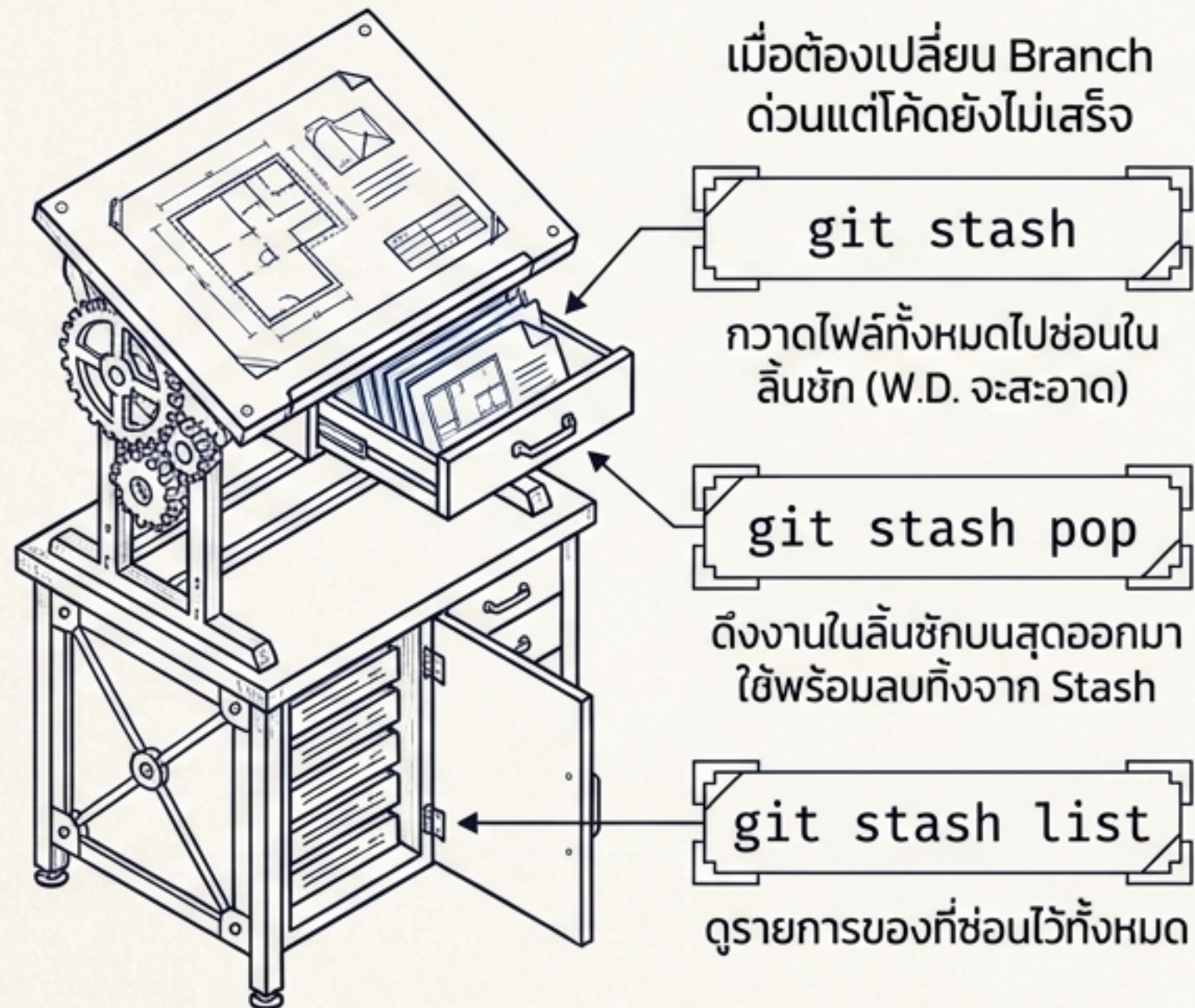
Quadrant 4

git reset --hard <commit>

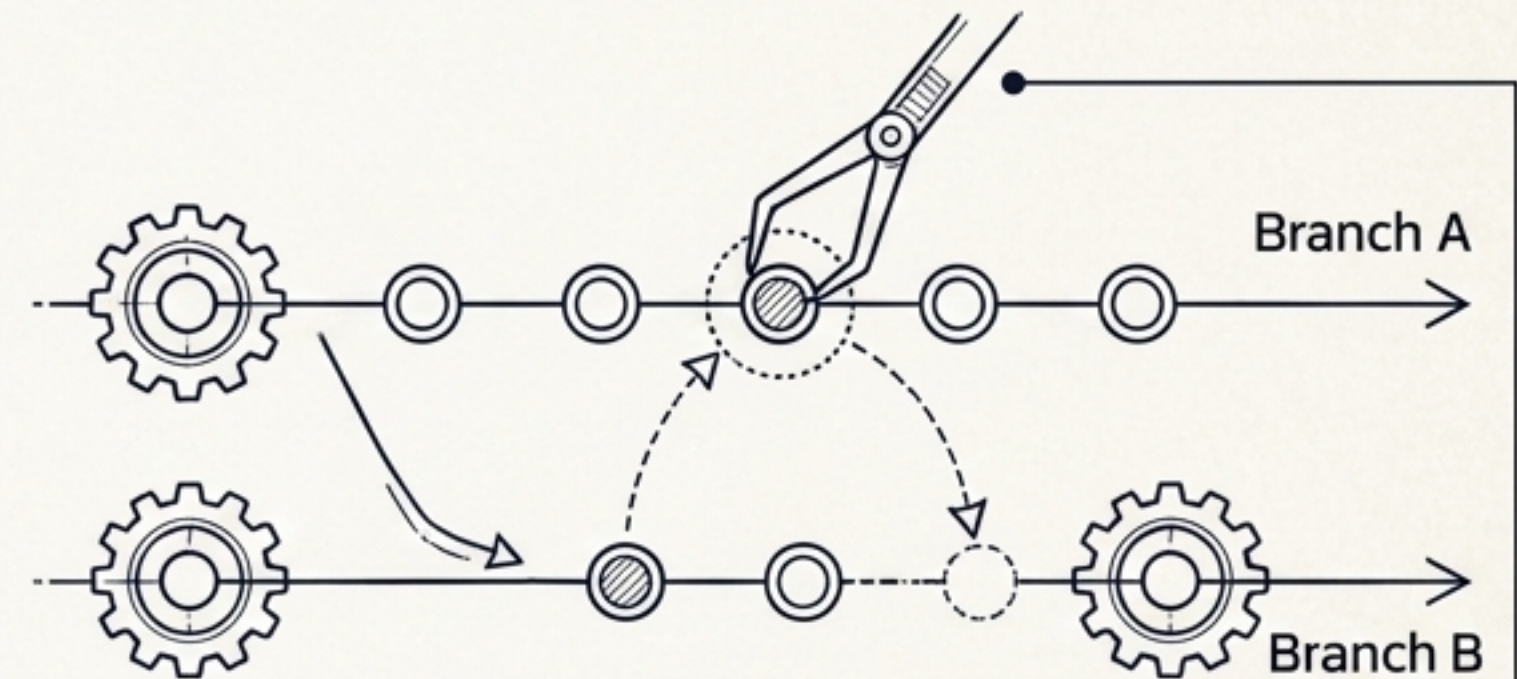
[อันตราย - ล้างไฟฟ์!] ย้อนทุกอย่างกลับไปตามอดีต โค้ดที่ไม่ได้เซฟหายถาวร

Interruptions: จัดการงานค้างคาและหยิบยืมโค้ด

STASH - ลั่นชักซ่อนงาน



CHERRY-PICK - คีบเฉพาะที่อยากได้

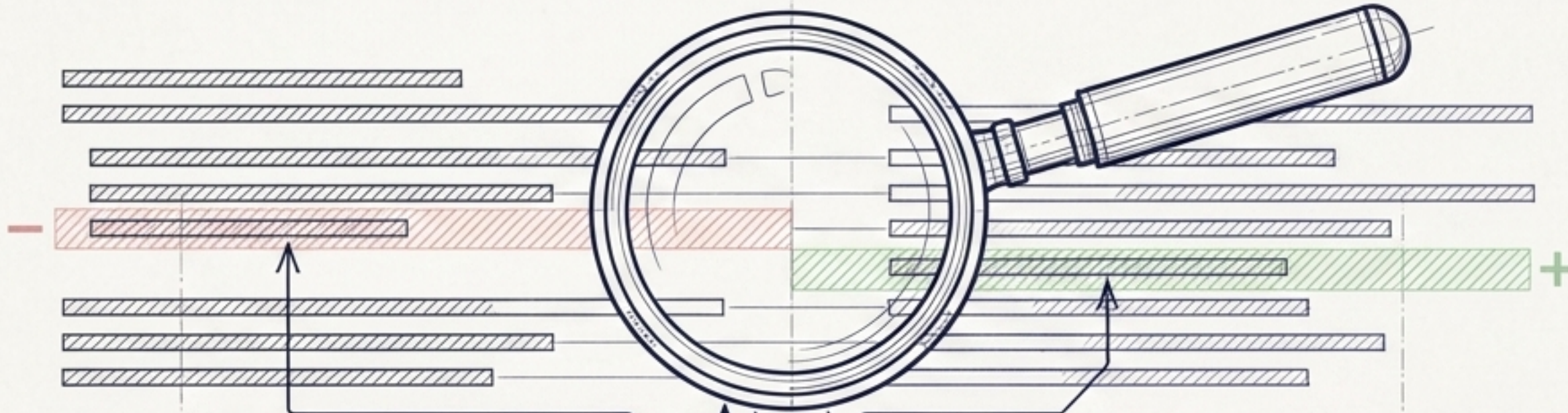


`git cherry-pick <commit>`

เล็งเฉพาะ Commit เด็ดๆ จาก Branch อื่น
ดึงเข้ามาใส่ Branch ปัจจุบัน โดยไม่ต้อง Merge ทั้งเส้น

Cleanup: `git clean -fd` (ลบไฟล์และโฟลเดอร์ขยะที่ไม่ได้ Track ทิ้ง)

Time Travel & Diffs: ส่องอดีตและความเปลี่ยนแปลง



VIEWING HISTORY (แกะรอยประวัติศาสตร์)

`git log --oneline` ◀
สรุปประวัติแบบกระชับ 1 บรรทัดต่อ 1 Commit

`git log --graph` ◀
ดูประวัติเป็นโครงสร้างต้นไม้

`git log @{1.month.ago}..HEAD` ◀
ดู Relative dates (เช่น ขอดูประวัติย้อนหลัง 1 เดือน)

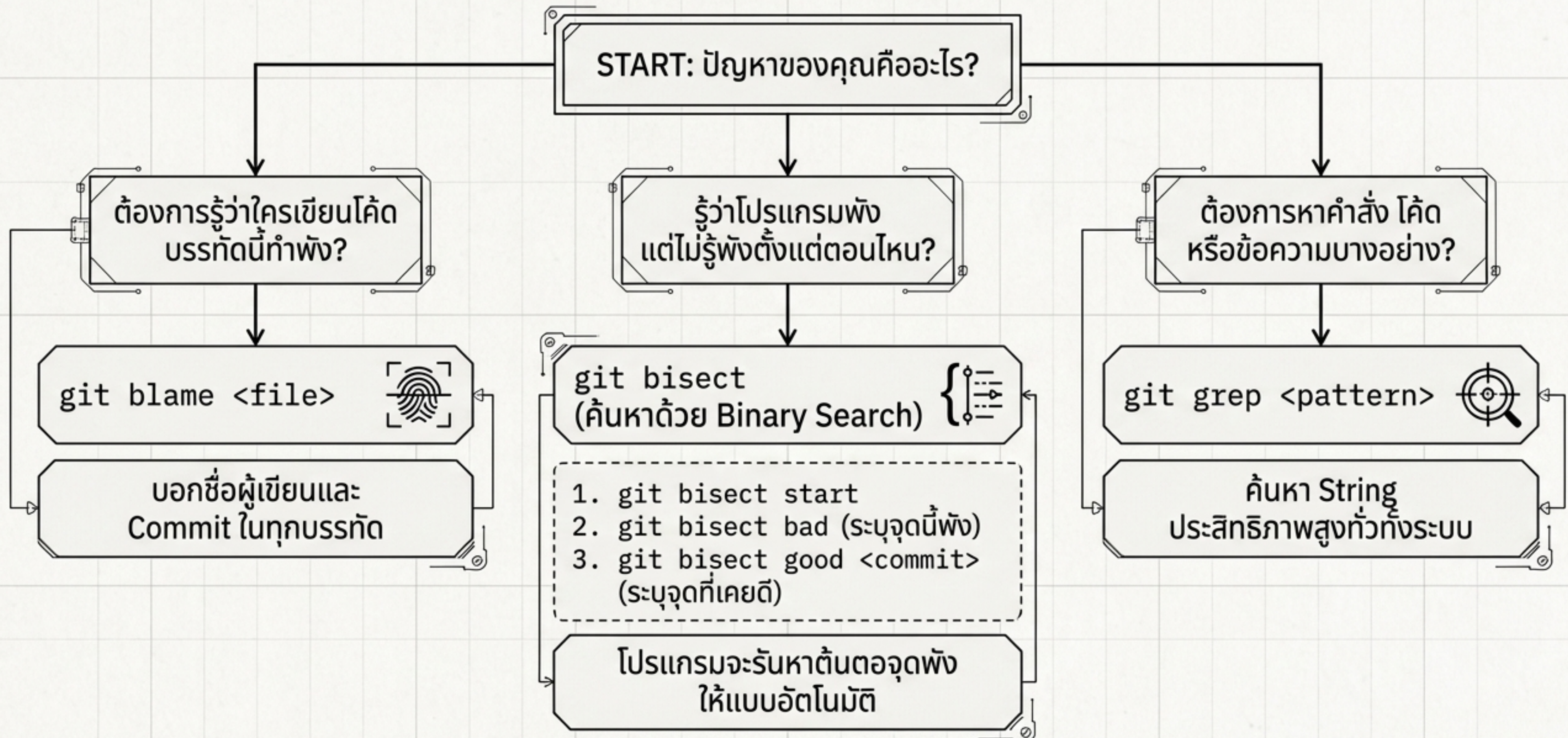
ANALYZING DIFFS (เปรียบเทียบจุดต่าง)

▶ `git diff`
ดูว่า W.D. ต่างจาก Staging อย่างไร

▶ `git diff --cached`
ดูว่า Staging เตรียมจะส่งอะไรไปเป็น Commit ใหม่

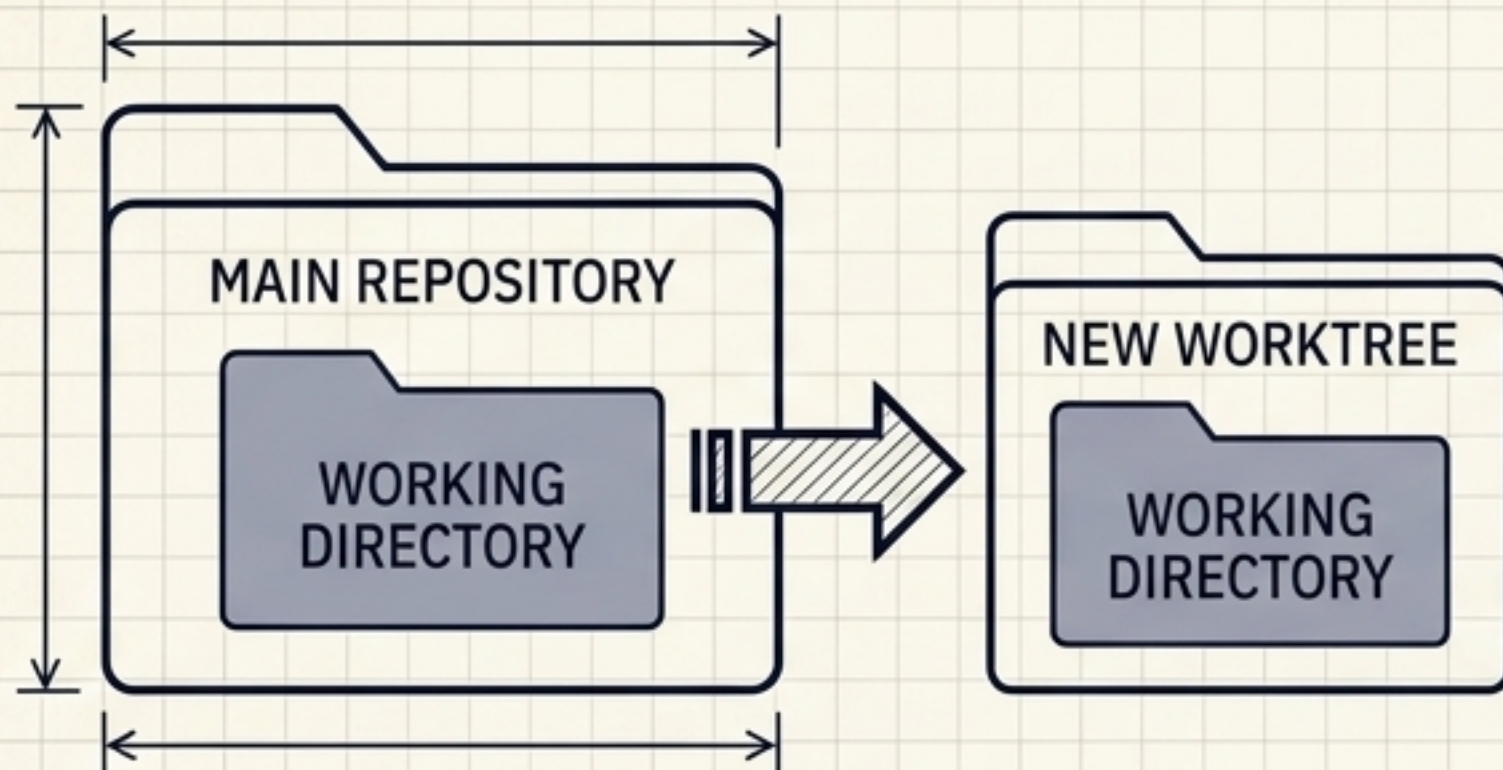
▶ `git diff <commit1> <commit2>`
เปรียบเทียบรหัส 2 จุดในอดีตอย่างละเอียด

The Detective Toolkit: แผนผังวินิจฉัยเมื่อเจอบั๊ก



Advanced Workspace: โครงสร้างโปรเจกต์ระดับสูง

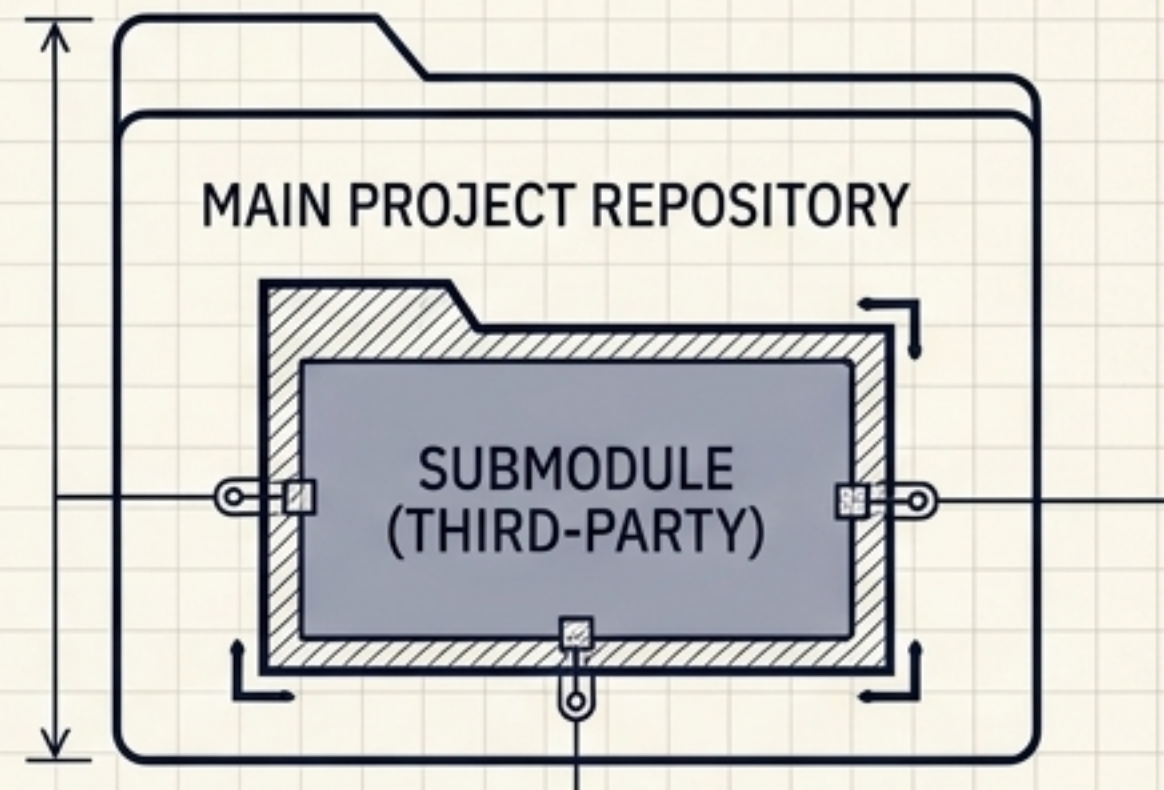
WORKTREES - ร่างแยกของโปรเจกต์



```
git worktree add <path> <branch>
```

→ แยก Working Directory ใหม่ออกไปอีกโฟลเดอร์ เพื่อทำงานกับ 2 Branches พร้อมกันได้โดยไม่ต้องคอย Switch สลับไปมา

SUBMODULES - โปรเจกต์ซ้อนโปรเจกต์



```
git submodule add <repo> <path>
```

→ ฝัง Repository อื่น (เช่น Third-party library) เข้ามาใน Repository หลัก โดยแยกการจัดการเวอร์ชันกันอย่างเป็นระเบียบ

```
git submodule update --remote
```

→ ดึงโค้ดล่าสุดจาก Subproject

Maintenance & Emergency: การซ่อมบำรุงและกู้ชีพ

EMERGENCY RECOVERY - กู้ชีพ



`git reflog`

- ฟีเจอร์ช่วยชีวิตที่บันทึก **‘ทุกการขยับตัว’** ของ HEAD ไม่ว่าจะเป็น Checkouts, Resets, หรือ Merges
- แม้คุณจะทำ Reset --hard ไป ก็สามารถเปิด Reflog หา Hash แล้วกู้ Commit ที่คิดว่าหายไปถาวรกลับมาได้!

DATA INTEGRITY - ดูแลฐานข้อมูล



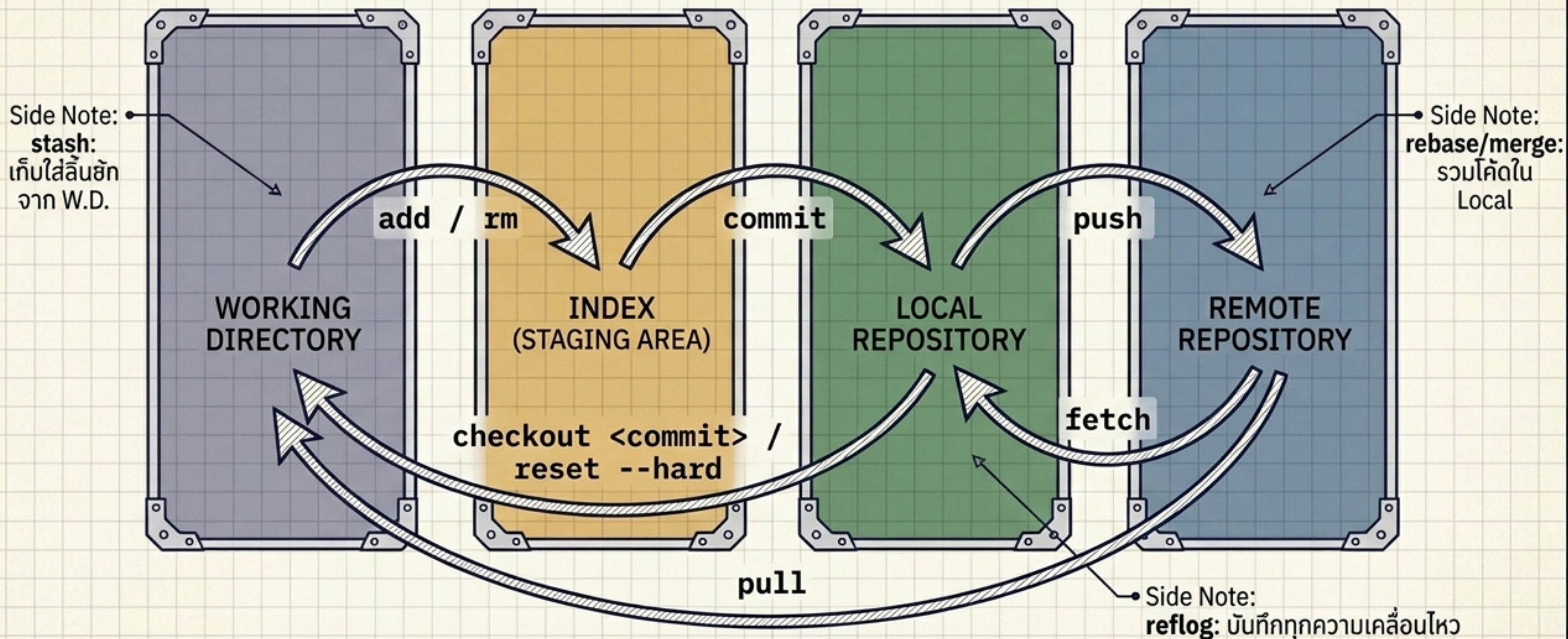
`git gc`

(Garbage Collection): บีบอัดไฟล์และลบ Object ชะงักที่เข้าถึงไม่ได้ เพื่อคืนพื้นที่ว่างให้ระบบ

`git fsck`

ตรวจสอบความสมบูรณ์และหาไฟล์ที่เสียหายใน Database (SHA-1 Integrity Check)

The Complete Git Map: พิมพ์เขียวสรุปคำสั่งในหน้าเดียว



บันทึกสไลด์นี้เก็บไว้เป็นคู่มือประจำโต๊ะทำงานของคุณ